
ProofDebt: When Does Memory Help Continual Software Repair?

Adarsh Agrawal
Senior Member, IEEE

Abstract

Coding agents are increasingly expected to work on the same repository over time, yet most evaluations treat every issue as a fresh start. This makes a basic question hard to answer: when should memory help? We introduce PROOFDEBT, a continual-repair protocol whose memory is built from native-test outcomes, changed files, dependency links, and patch status rather than self-written summaries. We evaluate 891 repair attempts across 11 three-step ChainSWE chains and three model families. Average effects are small under token-matched comparisons, but the average hides an important difference between tasks. Two coders independently audit all 22 later-step tasks and agree on 19 of 22 information-need labels before adjudication (86.4%; unweighted Cohen’s $\kappa = 0.77$). Their consensus audit identifies seven tasks where the correct repair genuinely needs information from an earlier repair. On these tasks, reset succeeds on 14.3% of attempts, PROOFDEBT on 60.3%, and an oracle memory condition on 82.5%. On the other 15 tasks, the three conditions remain within 4.5 points. A trace audit shows that practical ledgers usually contain the right obligation but often fail to present it as something the model must preserve. These results suggest that continual-agent evaluations should measure whether a task offers a real opportunity for memory before averaging memory effects.

1 Introduction

Software maintenance is sequential. A repair changes the code, tests, and assumptions that later repairs inherit. Most repository-repair benchmarks instead restore a clean snapshot for every issue. This makes individual tasks easy to compare, but it does not tell us whether an agent can maintain a repository over time.

Persistent memory seems like a natural solution. A coding agent could retain earlier patches, test outcomes, or a compact summary of the repository’s current obligations. But memory also adds prompt content, cost, and stale information. An average comparison across all later tasks is therefore hard to interpret. If a later task is independent of earlier work, memory has nothing useful to contribute.

We study this problem with PROOFDEBT, a checker-backed memory protocol for continual software repair. After each repair, the runner records facts derived from the repository and its native tests. The model can read this ledger, but it cannot write or erase the factual fields.

The first result is that average memory effects are small and unstable across matched campaigns. The more useful result appears after asking which tasks could benefit from history. Two coders audit all 22 later-step tasks without seeing model outcomes. Only seven tasks, drawn from four repository chains, require information established by an earlier repair. On this subset, PROOFDEBT improves repair success from 14.3% to 60.3%, while an oracle condition reaches 82.5%. The other 15 tasks show almost no separation.

This changes how we interpret continual-agent experiments. A memory method should not be judged only by an average over every later task. The evaluation should also report how often history is relevant, whether the memory captured the relevant fact, and whether the model understood it as a binding constraint.

The paper makes three contributions:

- a checker-backed protocol for carrying verified repository state across an ordered sequence of repairs;
- an outcome-blinded audit that measures how often later tasks truly need earlier repair information; and
- an opportunity-aware analysis that separates memory capture from memory use.

2 ProofDebt

2.1 Continual repair

Each repository chain contains three ordered issues. At every step, the model receives the current repository, the current issue, and the information allowed by its experimental condition. The proposed patch must apply cleanly and pass the repository’s native checker. Accepted code remains in the repository for the next issue.

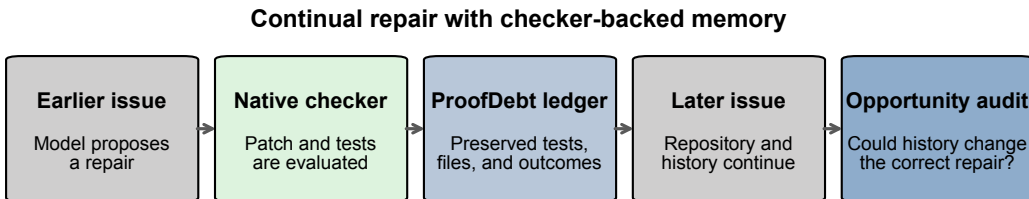


Figure 1: PROOFDEBT records checker-backed facts after each repair. Before interpreting a memory effect, the evaluation asks whether those facts could change the correct solution to a later issue.

2.2 Memory conditions

Reset. The model receives the current issue, the repository tree, and the same file hints used by the other conditions, but no earlier chain history.

Raw context. The model also receives earlier candidate patches, patch-application outcomes, native-test outcomes, and recent test-output excerpts.

ProofDebt. The model instead receives a compact ledger of preserved tests, changed files, dependency links, patch outcomes, and ordered repair history.

Oracle memory. For diagnosis, we include a condition that states the ground-truth obligations the next repair must preserve. It uses the same repair interface but provides a clearer version of the relevant historical constraint.

2.3 Checker-backed state

When a repair makes a previously failing regression test pass, that test becomes an obligation for later steps. The ledger tracks whether each obligation remains expected, passes, fails, errors, or is not collected. It also stores changed files, source-to-test links, patch status, and the ordered step history.

The model only proposes a diff. The runner applies the patch, runs the native checker, and creates the next ledger. A separate recomputation matched every stored ledger in the 891-attempt primary campaign.

2.4 When can memory help?

For each later issue, we ask four questions:

1. Was an earlier repaired obligation recorded?
2. Could that obligation still matter to the later issue?
3. Does the later repair depend on the earlier work?
4. Does the correct repair need information established by that work?

We call tasks satisfying the final question *high-confidence reuse opportunities*. This label describes the benchmark. It is not an online router because the audit uses gold patches and native-test evidence.

3 Study design

3.1 Experimental setup

We use a pinned ChainSWE [1] snapshot and replay each candidate chain sequentially with its hidden tests and gold patches. Eleven three-step chains from seven Python repositories pass sequential gold replay and form the primary cohort.

Table 1: Experimental setup at a glance.

Item	Setting
Cohort	11 repository chains, 3 ordered issues per chain
Models	Claude Sonnet 5, Qwen3 Coder 480B, GPT-5.6 Terra
Primary conditions	Reset, raw context, PROOFDEBT
Diagnostic condition	Oracle memory
Prompt variation	3 controlled prompt variants per model and condition
Primary scale	891 native-test-scored repair attempts
Later-task audit	22 unique tasks from steps 2 and 3
Main outcome	Patch applies and the repository’s native checker passes
Statistical unit	Repository chain

The three prompt variants change the reasoning instruction. They are not independent provider seeds. Full model and execution settings appear in Appendix A.4.

3.2 Matched comparison

The initial campaign gave PROOFDEBT more input tokens than reset. We therefore ran two additional campaigns with generic condition labels and matched token budgets. We pool the two campaigns for the average-effect comparison.

Confidence intervals use 20,000 hierarchical bootstrap draws over repositories and chains. Exact chain-level sign tests are corrected within each comparison family. Model, prompt, and task repetitions are not treated as independent samples.

3.3 Opportunity audit

After the campaigns were complete, two coders independently classified all 22 later-step tasks. They inspected issue text, gold patches, native-test outcomes, and static dependencies without viewing generated repairs or condition-level success. Before adjudication, they agreed on 19 of 22 information-need labels (86.4%; unweighted Cohen’s $\kappa = 0.77$) and 20 of 22 semantic-dependency labels (90.9%; $\kappa = 0.85$). Five field-level disagreements across four tasks were resolved by evidence-based consensus.

Seven tasks meet the strict information-need criterion. The subgroup was defined after the campaigns, so we treat it as an exploratory explanation of the aggregate result. The full labels and coding guide are included in the supplement.

4 Results

4.1 Average scores hide where memory helps

Table 2 summarizes the aggregate comparison. Neither raw context nor PROOFDEBT produces a clear average improvement over reset. The matched pooled estimates are small, and both confidence intervals include zero.

Table 2: Aggregate repair-success differences relative to reset.

Evaluation	Memory	Difference	Chain-level 95% CI
Initial campaign	Raw context	+1.0 pt	$[-3.4, +6.2]$
Initial campaign	PROOFDEBT	-1.7 pt	$[-5.4, +2.6]$
Matched pooled	Raw context	+2.2 pt	$[-0.3, +5.0]$
Matched pooled	PROOFDEBT	+1.9 pt	$[-1.7, +5.7]$

This average combines tasks where earlier information can change the repair with tasks where it cannot. The opportunity audit separates those cases.

4.2 Only seven later tasks need earlier information

Twelve of the 22 later tasks contain a recorded obligation from an earlier repair. Ten could plausibly reuse that obligation, nine show a semantic dependency, and seven require earlier repair information. The seven strict opportunities are concentrated in four repository chains.

This distinction matters because repeated models and prompts increase the number of attempts but do not create new opportunities. The relevant denominator is 22 unique later tasks, not 594 repeated attempts.

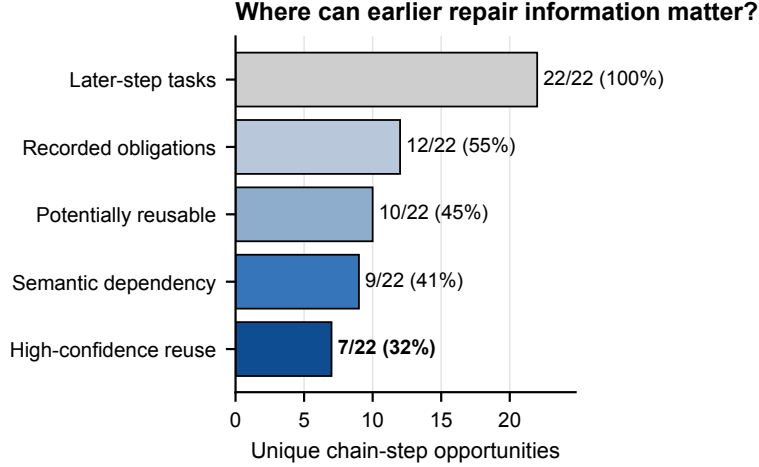


Figure 2: Seven of 22 later-step tasks provide a high-confidence opportunity to reuse information from an earlier repair.

4.3 Memory gains are concentrated on those tasks

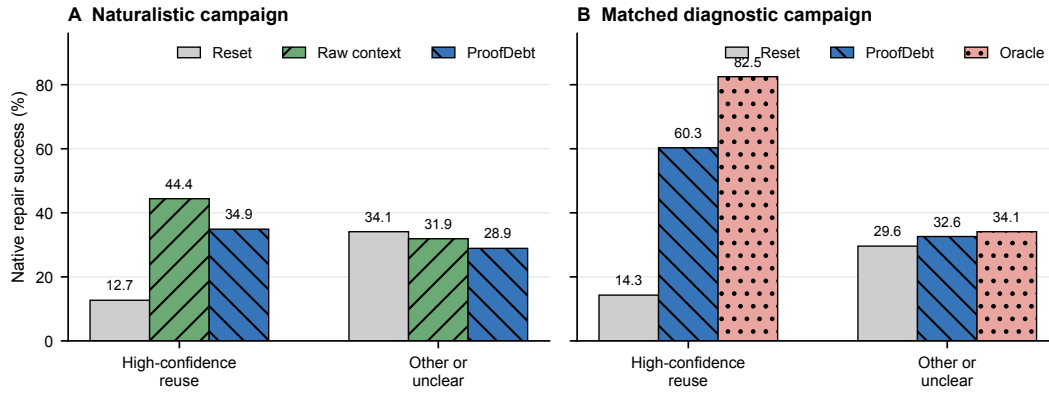


Figure 3: Repair success separated by whether the later task needs earlier information. Each high-confidence bar contains 63 attempts over seven unique tasks.

In the initial campaign, success on the seven high-confidence tasks rises from 12.7% under reset to 44.4% with raw context and 34.9% with PROOFDEBT. The other 15 tasks show no comparable improvement.

The matched diagnostic campaign produces a larger split. Reset succeeds on 14.3% of attempts, PROOFDEBT on 60.3%, and oracle memory on 82.5%. On the other 15 tasks, all three conditions remain within 4.5 points. The same pattern therefore appears under both the naturalistic and matched designs.

The strict subset spans only four chains from three repositories, so we report it descriptively rather than attaching a confirmatory confidence interval or p -value. Both memory conditions outperform reset in every chain. Leave-one-chain-out differences range from 44.4 to 46.7 points for PROOFDEBT and from 66.7 to 68.9 points for oracle memory (Appendix A.2).

4.4 The remaining gap is about actionability

In 9 of 10 inspected PROOFDEBT traces, the ledger captures the relevant obligation, and in 8 it presents that obligation to the model. Only four model responses acknowledge or try to preserve it, and two repairs succeed. The oracle states the constraint explicitly in all 10 traces, eight responses acknowledge it, and five repairs succeed.

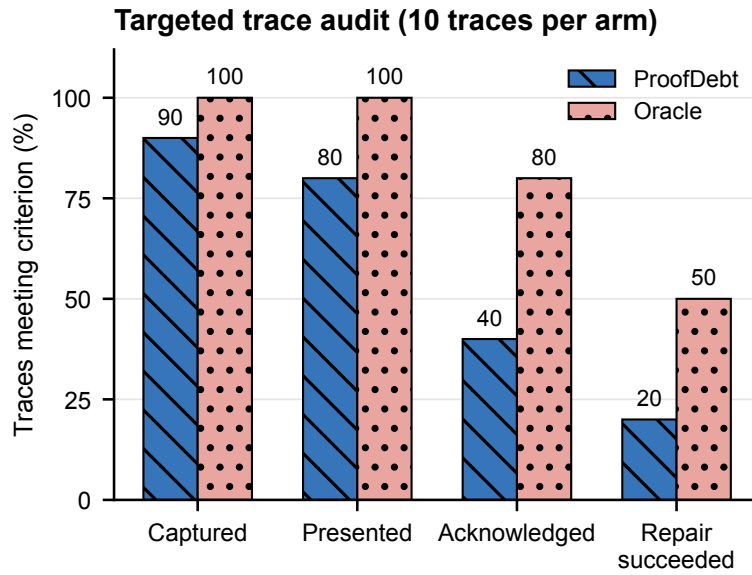


Figure 4: A targeted audit of 10 PROOFDEBT and 10 oracle traces. The practical ledger usually contains the right obligation, but the model does not always treat it as a requirement.

The practical problem is therefore not only whether memory stores the right fact. The representation must also make clear what the model is expected to preserve, and the model must implement that constraint correctly.

5 What this changes

The opportunity audit suggests a simple reporting rule for continual-agent benchmarks. Studies should report the number of unique later tasks, the number carrying earlier obligations, and the number whose correct solution actually depends on those obligations. Without these counts, an average memory effect can hide both useful and irrelevant cases.

The trace audit also suggests a concrete design improvement. A practical ledger should identify a small set of current obligations, connect them to the relevant files and tests, and state them as requirements rather than passive history. The oracle result shows the remaining room for improvement, but it does not establish that an automatic system can identify these constraints online.

Memory should eventually be selective. A useful router would decide from pre-repair information whether the current issue is likely to benefit from history. Building and prospectively testing such a router is left for future work.

6 Limitations

The study contains 11 short, Python-dominated chains, and the strict opportunity subset contains seven tasks from four chains. The opportunity audit was conducted after the campaigns, although the coders did not see model outcomes and reached high agreement. Gold patches make the audit useful for understanding the benchmark but unsuitable as an online routing method. The oracle receives privileged information, and the trace audit covers only 10 examples per condition. Finally, a passing native test suite is strong executable evidence for the benchmark task, but it is not a guarantee of complete semantic correctness.

7 Related work

SWE-bench [4] established executable issue resolution in real repositories. SWE-agent [6], Agentless [9], and RepoCoder [11] improve repository localization and editing but generally evaluate issues independently. ChainSWE [1] instead organizes issues into ordered repository lifetimes.

Reflection and memory systems retain critiques, summaries, or retrieved episodes [14, 16, 19]. PROOFDEBT differs by restricting factual memory to checker-derived evidence. Related systems use proof assistants, static analysis, or contracts to guide code generation [21, 24, 26]. Our focus is different: we ask when historical state has a real path to improving a later repair.

8 Conclusion

Continual coding agents need more than a place to store history. They need an evaluation that distinguishes useful history from irrelevant history. PROOFDEBT provides checker-backed repository memory, while the opportunity audit shows that only seven of 22 later tasks genuinely need earlier repair information. Memory produces large gains on those tasks and little change elsewhere. The next step is to turn captured obligations into clear, selective, and actionable constraints.

References

- [1] Qirui Jin et al. ChainSWE: Benchmarking coding agents on multi-bug software maintenance. *arXiv preprint arXiv:2607.02606*, 2026.
- [2] Shashank Indukuri and Adarsh Agrawal. Grounded optimization: A layered engineering framework for reducing llm hallucination in automated personal document rewriting. *arXiv preprint arXiv:2607.01457*, 2026.
- [3] AAG Intelligence. Amazon nova multimodal embeddings: Technical report and model card. Technical report, Amazon, 2025.
- [4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- [5] Bertie Vidgen, Adarsh Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. Introducing v0.5 of the ai safety benchmark from mlcommons. *arXiv preprint arXiv:2404.12241*, 2024. Google Scholar primary record.
- [6] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- [7] Adarsh Agrawal and Rahul Suresh Babu. Incident memory: Training-free operational memory through sequential pattern mining and velocity-stratified retrieval. *arXiv preprint arXiv:2609.01616*, 2026.
- [8] AAG Intelligence. Amazon nova sonic: Technical report and model card. Technical report, Amazon, 2025.
- [9] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint*, 2024.
- [10] Adarsh Agrawal and Shashank Indukuri. Schema-first retrieval: Embedding catalogs for natural language analytics. *arXiv preprint arXiv:2606.28387*, 2026.
- [11] Yizhuo Zhang et al. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Conference on Empirical Methods in Natural Language Processing*, 2023.
- [12] Bertie Vidgen, Adarsh Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. others. 2024. introducing v0.5 of the ai safety benchmark from mlcommons. *arXiv preprint arXiv:2404.12241*, 2024. Separate Google Scholar record.
- [13] AAG Intelligence. Amazon nova premier: Technical report and model card. Technical report, Amazon, 2025.
- [14] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- [15] Wei Feng, Adarsh Agrawal, Haifeng Ling, Erik Blasch, E. Adiles-Cruz, P. T. Schrader, and J. Wei. DDDAS probability learning for natural disaster change detection. In *International Conference on Dynamic Data Driven Applications Systems*, pages 90–99, 2024.
- [16] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [17] Rahul Suresh Babu and Adarsh Agrawal. Self-healing agentic orchestrators for reliable tool-augmented large language model systems. *arXiv preprint arXiv:2606.01416*, 2026.
- [18] AAG Intelligence. Amazon nova 2: Multimodal reasoning and generation models. Technical report, Amazon, 2025.

- [19] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology*, 2023.
- [20] Adarsh Agrawal and J. Li. Mitigating bias in ai using debias-gan. NVIDIA GTC Conference, 2022.
- [21] Yuwei Liu et al. KVerus: Scalable and resilient formal verification proof generation for Rust code. *arXiv preprint arXiv:2605.03822*, 2026.
- [22] Bertie Vidgen, Adarsh Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. Introducing v0.5 of the ai safety benchmark from mlcommons, 2024. <https://arxiv.org/abs/2404.12241>, 2024. Separate Google Scholar URL record.
- [23] AAG Intelligence. The amazon nova family of models: Technical report and model card. Technical report, Amazon, 2024.
- [24] Zhe Ye et al. Vero: Can AI agents build formally verified software repositories? *arXiv preprint arXiv:2608.13522*, 2026.
- [25] A. V. Adarsh Agrawal and Nikhil Garg. Time series forecasting of producer price index, using arima. *International Journal of Science and Research (IJSR)*, 7(7):1019–1024, 2018.
- [26] Jim Woodcock, Gabriel Leite, Augusto Sampaio, and Ran Wei. CAPRI: Contract-aware proof repair for Isabelle. *arXiv preprint arXiv:2608.13459*, 2026.
- [27] Bertie Vidgen, Adarsh Agrawal, A. M. Ahmed, V. Akinwande, N. Al-Nuaimi, N. Alfaraj, et al. How to construct ai safety benchmarks, n.d. Google Scholar record; publication year not listed.

A Technical appendix

A.1 Opportunity-audit summary

Table 3: Opportunity-audit counts over the 22 unique later-step tasks.

Audit stage	Tasks
Later-step tasks considered	22
Earlier repaired obligation recorded	12
Obligation potentially reusable	10
Semantic dependency on earlier work	9
High-confidence information need	7

The supplement contains the complete task-level labels. Each row uses the following fields:

Table 4: Data dictionary for the opportunity labels.

Field	Meaning
<code>recorded_fail_to_pass</code>	Whether an earlier repaired regression test is recorded for the chain.
<code>semantic_dependency</code>	Whether the later repair depends on behavior introduced or constrained by earlier work.
<code>information_need</code>	Whether the correct later repair needs information established by the earlier repair.
<code>evidence</code>	A short explanation grounded in issue text, gold patches, native tests, and static dependencies.

A.2 Matched diagnostic subgroup

The seven high-confidence tasks were evaluated in a separate matched diagnostic campaign. They come from four chains in three repositories, with nine attempts per task and condition.

Table 5: Chain-level outcomes for the seven-task diagnostic subset. Counts are successful repairs over attempts in each condition.

Chain	Tasks	Reset	PROOFDEBT	Oracle
docformatter (2022-08)	2	2/18	11/18	15/18
docformatter (2023-11)	1	1/9	5/9	7/9
Python-Markdown (2023-03)	2	3/18	11/18	15/18
brutils-python (2023-03)	2	3/18	11/18	15/18
Total	7	9/63	38/63	52/63

The pooled descriptive differences are +46.0 points for PROOFDEBT versus reset, +68.3 points for oracle versus reset, and +22.2 points for oracle versus PROOFDEBT. All four chains favor both memory conditions over reset.

These checks show that no single chain creates the descriptive pattern. However, three repositories are too few for a stable estimate of cross-repository uncertainty, so we do not treat this subgroup as confirmatory.

Table 6: Leave-one-chain-out sensitivity of the diagnostic differences. Values are percentage-point changes relative to reset.

Dropped chain	PROOFDEBT – Reset	Oracle – Reset
docformatter (2022-08)	+44.4	+66.7
docformatter (2023-11)	+46.3	+68.5
Python-Markdown (2023-03)	+46.7	+68.9
brutils-python (2023-03)	+46.7	+68.9

A.3 Inter-coder reliability and adjudication

Two study-team researchers independently coded all 22 later-step tasks using the written rubric in Table 4. Coding was post hoc. The coders could inspect issue text, gold patches, native-test outcomes, and static dependencies, but not generated repairs or condition-level success. We compute standard unweighted Cohen’s κ from the pre-adjudication labels. The two substantive fields use the ternary categories yes, unclear, and no.

Table 7: Independent agreement before adjudication. The information-need field defines the strict seven-task subset.

Field	Exact agreement	Cohen’s κ
Recorded fail-to-pass	22/22 (100%)	1.00
Semantic dependency	20/22 (90.9%)	0.85
Information need	19/22 (86.4%)	0.77

Table 8: Pre-adjudication contingency tables. Rows are Coder A labels and columns are Coder B labels.

Information need				Semantic dependency			
	Yes	Unclear	No		Yes	Unclear	No
Yes	6	1	0	Yes	8	0	1
Unclear	1	2	0	Unclear	1	2	0
No	0	1	11	No	0	0	10

There were five field-level disagreements across four unique tasks. Each coder presented evidence from the issue, gold patch, native tests, and static dependencies. If discussion did not resolve the label, the coders reran the native test while substituting the relevant earlier source state to determine whether the earlier repair changed the later task’s behavior. Consensus was required; no majority-vote fallback was used. The five disagreements were resolved in one 90-minute session.

These values show that the two coders applied the rubric consistently within this audit. They do not establish independent external validity because both coders were members of the research team.

Table 9: Disagreements and consensus resolutions. The same Click task produced one disagreement in each substantive field.

Field	Task	Coder A	Coder B	Resolution and evidence
Semantic dependency	click_20230518_step2	Unclear	Yes	Yes: the option parser shares a state machine, and the earlier repair changes valid transitions.
Semantic dependency	brutils-python_20230627_step3	Yes	No	No: the later change is an independent utility function without shared state.
Information need	click_20230518_step2	No	Unclear	Unclear: components overlap, but the need for historical information is uncertain; the task is excluded from the strict subset.
Information need	docformatter_20221228_step3	Unclear	Yes	Yes: the later gold patch preserves an indentation rule established by the earlier repair, and removing that rule breaks the later test.
Information need	Python-Markdown_20191125_step3	Yes	Unclear	Yes: the later crash occurs only when parser state introduced by the earlier repair persists.

A.4 Reproducibility details

The primary campaign contains 891 scored repair attempts and 297 model-condition-prompt chain trajectories. It made 1,225 model calls using approximately 11.9 million input tokens and 1.8 million output tokens. No model training was performed.

Table 10: Model and generation settings used in the reported campaigns.

Setting	Value
Models	Claude Sonnet 5: <code>global.anthropic.claude-sonnet-5</code> Qwen3 Coder 480B: <code>qwen.qwen3-coder-480b-a35b-v1:0</code> GPT-5.6 Terra: <code>openai.gpt-5.6-terra</code> .
Provider	AWS Bedrock in <code>us-west-2</code> ; Runtime Converse for Claude and Qwen, and Bedrock Mantle Responses for GPT-5.6 Terra.
Sampling	Temperature, top-p, provider seed, stop sequences, and reasoning controls were omitted, so model-specific provider defaults applied. Campaign labels 404–406 denote controlled prompt variants, not provider sampling seeds.
Generation limits	4,000 output tokens per call; one primary generation and at most one format-repair generation.
Context construction	No additional model-context token cap; at most 30,000 source characters per file and 360 repository-tree entries were placed in a prompt.
Failure handling	Up to five invocation attempts with 0.75, 1.5, 3, and 6 second backoffs. No explicit client timeout was configured; the longest recorded call was 134 seconds. A still-invalid patch after one format repair was recorded as malformed or unapplied.

Table 11: Frozen repository-checker environments. Task-specific pytest selectors are retained in the supplemental manifest.

Repository	Python	pytest
flake8-simplify	3.12.13	9.1.1
docformatter	3.12.13	9.1.1
Python-Markdown	3.12.13	9.1.1
brutils-python	3.12.13	9.1.1
Click	3.12.13	9.1.1
dataclasses-jsonschema	3.9.25	7.4.4
freezegun	3.9.25	7.4.4

The runner maintained one frozen virtual environment per repository and materialized a fresh source workspace for each chain trajectory. Native commands ran from the repository root with the workspace on `PYTHONPATH`. Each invocation had a 300-second hard timeout; none of the 891 primary checks timed out. Exact dependency freezes and task-specific pytest commands are retained with the campaign manifest.

A.5 Compute

Model inference used managed APIs, while repository checks ran on a 64-vCPU Intel Xeon Platinum 8375C host with 256 GB RAM and Ubuntu 24.04. The runner used at most 18 concurrent chain workers. Provider-side accelerator details were not disclosed.

The 891-attempt primary campaign took approximately 6–7 hours. Each matched diagnostic campaign took approximately 8 hours, and the reported campaign and audit work totaled approximately 30–35 hours. Recorded model latency had a median of 8.7 seconds, a 95th percentile of 76.4 seconds, and a maximum of 134.2 seconds. Across the 891 native-checker logs, checker runtime had a median of 0.50 seconds, a 95th percentile of 1.38 seconds, and a maximum of 2.52 seconds. The study required no model training and no dedicated client-side GPU allocation.

A.6 Artifact availability

The anonymized supplement accompanying the submission will include the manuscript source, unmodified workshop style, opportunity labels for all 22 later-step tasks, coding rubric, 20-row trace audit, task-level outcome tables, figure and analysis scripts, frozen environment specification, and a chain manifest with repository revisions. These scripts regenerate the paper’s tables and figures without model access.

Full prompts, model responses, candidate patches, ledger snapshots, and native checker logs are larger and will be available through an anonymous review archive. Re-running model inference requires ChainSWE access and AWS Bedrock credentials; the recorded campaign entry point is `./run_project.sh campaign`. The primary campaign takes roughly 6–7 hours with 18 workers.

A.7 Existing assets and licenses

Table 12: Existing software and benchmark assets used by the study.

Asset	License or terms
ChainSWE	MIT for metadata and task definitions; the exact pinned revision and checksum are recorded in the supplemental manifest. Underlying repositories retain their original licenses.
flake8-simplify	MIT
docformatter	MIT/Expat
Python-Markdown	BSD-3-Clause
brutils-python	MIT
Click	BSD-3-Clause
dataclasses-jsonschema	MIT
freezegun	Apache-2.0
Opportunity labels, trace audit, and analysis code	Apache-2.0
Coding rubric	CC-BY-4.0

Hosted models were accessed through AWS Bedrock under the applicable provider terms. No model weights are redistributed.

A.8 Broader impacts

Opportunity-aware evaluation can reduce the cost and distraction of using persistent memory on tasks that do not depend on history. Checker-backed state can also reduce fabricated or self-certified memory. At the same time, a ledger can create false confidence when its constraints are stale or ignored. The protocol should complement repository review and native testing rather than replace them.